

Programmation Fonctionnelle

TD n° 2 : Listes

Exercice 1 : Déterminer le type des expressions suivantes. Vérifier votre réponse sur machine.

```
- let e = 4 in (e, "a") :: []
- (fun x y -> x y) (fun l -> 1::l) [4; 8]
- fun l -> match l with [] -> 111 | a::_ -> a
```

Exercice 2 : Écrire les fonctions suivantes sur les listes. Les fonctions précédées de ◦ sont prédéfinies en Caml (elles s'écrivent `List.length`, `List.rev`, etc) et le but est de retrouver leur implémentation.

- `length` : 'a list -> int : calculer la longueur d'une liste.
Exemple : `length [1; 2; 3] = 3`.
- `list_sigma` : int list -> int : calculer la somme des éléments d'une liste d'entiers.
Exemple : `list_sigma [1; 2; 3] = 6`.
- `append` : 'a list -> 'a list -> 'a list : concaténer deux listes - cette fonction ne doit pas utiliser l'opérateur @.
Exemple : `append [1; 2; 3] [4; 5] = [1; 2; 3; 4; 5]`.
- `split` : ('a * 'b) list -> 'a list * 'b list : transformer une liste de paires en une paire de listes.
Exemple : `split [(2, 'a'); (1, 'b'); (3, 'c')] = ([2; 1; 3], ['a'; 'b'; 'c'])`.
- `rev` : 'a list -> 'a list : inverser l'ordre des éléments d'une liste.
Exemple : `rev [1; 2; 3] = [3; 2; 1]`.
- `flatten` : aplanir d'un niveau une liste de listes.
Exemple : `flatten [[2]; []; [3; 4; 5]] = [2; 3; 4; 5]`.
- `mem` : 'a -> 'a list -> bool : déterminer si une liste contient une valeur donnée.
Exemple : `mem 27 [12; 27; 1] = true`, `mem 3 [12; 27; 1] = false`.
- `simplify` : 'a list -> 'a list : renvoyer la liste obtenue en ne gardant que la première occurrence de chaque élément.
Exemple : `simplify ["ab"; "cd"; "ab"; "ef"; "ab"; "ef"] = ["ab"; "cd"; "ef"]`
- `successors` : 'a -> 'a list -> 'a list : la fonction telle que `successors x l` renvoie la liste des éléments situés après chacune des occurrences de `x` dans `l`. Exemples :

```
successors "cd" ["ab"; "cd"; "ab"; "ab"; "ef"] = ["ab"]
successors "ab" ["ab"; "cd"; "ab"; "ab"; "ef"] = ["cd"; "ab"; "ef"]
successors "ef" ["ab"; "cd"; "ab"; "ab"; "ef"] = []
```
- `list_min` : 'a list -> 'a : renvoyer le minimum d'une liste lorsque celle-ci est non vide, et lever une exception sinon (par échec du `match`, ou mieux, à l'aide de `failwith`).

Exercice 3 : Les fonctions suivantes sont dites d'ordre supérieur : l'un au moins de leurs arguments doit être une valeur fonctionnelle.

- `map` : `('a -> 'b) -> 'a list -> 'b list` : étant donnée une fonction et une liste, renvoyer la liste obtenu en appliquant cette fonction à chaque élément de la liste. On a donc `map f [a1; ...; an] = [(f a1); ...; (f an)]`.

Exemples :

```
map (fun x -> x + 1) [0; 3; 2; 4] = [1; 4; 3; 5]
map (fun x -> x = 0) [1; 0; 3; 0] = [false; true; false; true]
```

- `exists` : `('a -> bool) -> 'a list -> bool` : étant donné un *prédicat* `p` sur les éléments d'une liste (une fonction du type des éléments de la liste vers les booléens) et une liste `l`, renvoyer `true` si au moins un élément de `l` *satisfait* le prédicat `p`, c'est-à-dire si `l` contient au moins un élément `x` tel que `(p x)` soit égal à `true`.

Exemples :

```
exists (fun x -> x > 0) [0; -1; 2; 3] = true
exists (fun x -> x < 0) [0; -1; 2; 3] = false
```

- `for_all` : `('a -> bool) -> 'a list -> bool` : étant donné un prédicat `p` et une liste `l`, renvoyer `true` si *chaque* élément de `l` satisfait le prédicat `p`.

Exemples :

```
for_all (fun x -> x > 0) [0; -1; 2; 3] = false
for_all (fun x -> x <> 1) [0; -1; 2; 3] = true
```

- `filter` : `('a -> bool) -> 'a list -> 'a list` : étant donné un prédicat `p` et une liste `l`, renvoyer la liste de éléments de `l` vérifiant le prédicat `p`.

Exemples :

```
filter (fun x -> x > 0) [0; -1; 2; 3] = [2; 3]
filter (fun x -> x > 4) [0; -1; 2; 3] = []
```

- `find` : `('a -> bool) -> 'a list -> 'a` : étant donné un prédicat `p` et une liste `l`, renvoyer le premier élément `x` de `l` tel que `(p x)` soit égal à `true` si un tel élément existe, et lever une exception sinon.

Exemple : `find (fun x -> x < 0) [0; 1; -2; 0; -1] = -2`.

- `partition` : `('a -> bool) -> 'a list -> 'a list * 'a list` : étant donné un prédicat `p` et une liste `l`, renvoyer la paire formée : à gauche, des éléments de `l` satisfaisant `p`; à droite, des éléments de `l` ne satisfaisant pas `p`.

Exemple :

```
partition (fun x -> x > 2) [-3; 4; -1; 4; 3] = [4; 4; 3], [-3; -1]
```